GHOSTWIRE

Threat intelligence, told as narrative.

#ISSUE 004 OCTOBER 2026

The Proxy Harvest

A bandwidth-harvesting SDK that outlived a seizure, and the storefront its domains point to

// INTELLIGENCE FILE — GW-004-INF //

SUBJECT:	**AltPop / HopMon residential-proxy SDK**
CARRIER:	**Flix Vision (flix.com.vision) — off-store streaming app**
FAMILY:	**Popa / HopMon — operator unattributed**
SAMPLE:	**libalt.so 12.0.30 — arm64, Flix Vision 3.9.0**
PRESENT:	**v3.7.0 (10 Jun 2026), unchanged through 3.9.0**
STATUS:	**Live proxy path confirmed; corporate link inferred**

4	6	10-MIN	39
STOREFRONT DOMAINS	RELAY COMMANDS	LIVE RUN	HOSTS RELAYED

The Popa/HopMon proxyware documented by Synthient and Qurium did not stop after the 2 July 2026 FBI seizure. Its engine, renamed AltPop, was already inside Flix Vision, an unofficial free streaming app, three weeks earlier, and after the seizure the app swapped its burned command server for dynamic-DNS seeds. In a live run, an unmodified install given consent turned a cloud VM into a proxy exit for ten minutes. The app's hardcoded domains lead to ABNetworks, a storefront selling “residential IPs for AI data collection” whose own page says it sources them from “partner apps.” That ABNetworks monetises this app's

> bandwidth is inference from shared infrastructure and the vendor's own words, not a contract.

A bandwidth-harvesting SDK kept shipping inside a free streaming app through the 2026-07-02 law-enforcement seizure of its own operator's infrastructure, and its hardcoded domains point at a residential-proxy storefront whose own page says it sources IPs from consent-bundled apps.

The story

To watch a film on Flix Vision, you answer one question. Keep the app free by sharing a little unused bandwidth, or sit through ads instead. Most people tap the first option without reading the second line, and in that moment the phone stops being only a media player. It becomes a doorway, and strangers begin routing their web traffic through it, with the owner's home address on the outside.

That doorway is built by a proxyware SDK researchers have chased for a year. Synthient found it hidden inside X-Plore, a popular Android file manager, and Qurium mapped the relay network behind it. On 2 July 2026, the FBI and IRS-CI seized that network. The operation did not stop. Its engine, lightly renamed from `AlphaSdkEngine` to `AltSdkEngine` and branded AltPop, was already riding inside Flix Vision three weeks before the seizure. Within weeks the app had thrown away its burned command server and wired in a fresh one, and it kept harvesting. Takedowns move inventory more often than they end a business.

We watched it work. On a throwaway cloud machine, an unmodified copy of the app, with consent given, loaded its proxy library, dialed a controller, and for ten minutes relayed thirty-nine strangers' web requests out through our connection. For that window, our IP address was the one that would have landed in other people's logs.

The app does not hide where it reports. Its hardcoded domains lead to ABNetworks, a company with a Ras Al Khaimah mailbox, no named people, and a website advertising "one million residential IPs for AI data collection." The vendor's own transparency page says where those IPs come from: "partner apps" that give users something free in exchange for a slice of their connection. It never names Flix Vision. Whether ABNetworks runs the harvesting or simply buys from whoever does is the one thing the infrastructure cannot settle by itself, and the next section says so plainly.

What we did not show

What this investigation did not establish:

- That ABNetworks FZ-LLC operates the SDK, rather than being a customer, licensee or reseller of whoever does. The register field `publisher=flixview3` shows the SDK is sold per publisher, so there are several distinct roles here (SDK vendor, app publisher, proxy reseller, proxy customer) that the evidence does not separate.
- That our specific exit IP ever appeared in ABNetworks' product. We did not purchase or test it.
- Which party dispatched the live proxy job. The controller address arrived from the operator API, not from the app package, and we did not trace it back to the ABNetworks seed.

- Any named human behind the operation.

Why you should care

Here is the trade on offer. A free app, in exchange for the right to rent out your home connection to people you will never see. Whatever they do with it, scrape a site, hammer a login page, something worse, wears your IP address, not theirs. The consent screen is genuine, but it is one dialog between a person and the movie they opened the app to watch, and the business runs on the safe bet that they tap through it.

For a company, the risk walks in on an employee's personal phone sitting on a home or guest network, which quietly becomes an exit node for unknown third parties. "Residential IPs for AI data collection" is the polite name for the product: ordinary home addresses that anti-bot systems trust, rented out so that commercial scraping does not look like scraping.

How else this could be leveraged

[assessment, not observed in this case.] None of this is tied to streaming. The pattern, a harvesting SDK behind a one-tap consent screen and a tidy corporate storefront to sell the yield, fits any free app with reach: a game, a wallpaper pack, a flashlight. The relay protocol will open a connection to any host and port, so the same exit that scrapes a retailer today can carry something uglier tomorrow. And the storefront is cheap to stand back up: a no-code site, a free-zone company, a new set of domains. That is exactly what we watched happen after July.

SECTION 1

Background and scope

This issue continues the Popa/HopMon thread from Synthient's and Qurium's X-Plore research and from GhostWire Issue 003, into the period around the 2026-07-02 seizure of its operator's infrastructure. It covers three layers: the code lineage of the SDK, the enrollment behaviour and relay protocol, and the corporate infrastructure the app's domains lead to. The surrounding reseller market is noted for context only.

Method: static reverse engineering (radare2, Ghidra) of the native engines; a fully offline dynamic lab (a Linux network namespace with no route out, an x86_64 Android 14 emulator, and fake DNS, operator and relay servers) for the enrollment chain, where every server value was supplied by us; and one time-boxed live run of the unmodified app through an Oracle exit VM for ground truth. The corporate layer is passive OSINT (RDAP, certificate transparency, passive DNS, VirusTotal, and Microsoft's public identity endpoints). Interactions with operator infrastructure were limited to the live run (the app's own traffic for ten minutes) and to passive web requests; no account, purchase or authenticated access was used. The live run is detailed, with its harm-minimisation controls, below.

SECTION 2

Technical analysis

Code lineage (high-confidence)

The SDK engine class renames across versions while keeping its structure: `io.nn.alpha...AlphaSdkEngine` in the X-Plore era, then `io.nn.neunative...SdkEngine` (Neupop), then `com.amz.appsuite...SdkEngine` (Flix appsuite 3.6.3), then `com.ax.altpop...AltSdkEngine` (AltPop 3.7.0 and later). Confidence on the shared codebase is high (shared strings, a shared TLV protocol shape, a shared class skeleton, and the shared relay domains below). The generations also differ, so this is evolution, not a pure rename: the native API changed (`startSdk1`/`getUUID1` in the Alpha era versus `getParam`/`setParam` in AltPop), the protocol constant is 1 in the older library and 3 in AltPop, and a registration timer present in Alpha is gone. Direction and ownership are not established: this does not show that Flix Vision's developer is the same party as X-Plore's SDK vendor, and the rename order is from accumulated observation rather than a dated build captured at each hop. The original vendor of this SDK family, identified to us in correspondence with the developer of an earlier carrier app, is the subject of continuing work and is not detailed here.

The strongest lineage anchor is the relay list. AltPop's `libalt.so` hardcodes four load-balancer domains as its `lb.` entry points, and all four appear in Qurium's documented Popa relay set, which ties this engine to the original Popa operator infrastructure.

Native engine versions: `libalt.so` carries the plaintext string `12.0.30`. The arm64 build is byte-identical across Flix Vision 3.7.0, 3.8.0 and 3.9.0 (the armv7 build is likewise identical across those versions), so the native engine was not updated between 2026-06-10 and 2026-10-09. The arm64 and x86_64 builds are different binaries (different hashes), as expected. The X-Plore era `libnativesdk.so` carries `8.2.71` near its engine strings (moderate; adjacency only, and only two dated version points exist, so this is not a series). The X-Plore 4.43.14 build is wrapped with pairip (`libpairipcore.so`). The host-app proxy library is `libhopmn.so`, self-identified as `hopmnproxy-0.9.3.2`, which looks 3proxy-derived (moderate, from its banner and strings; source not diffed).

Signers differ across builds: the Flix Vision GitHub builds are self-signed (CN=FV), and Synthient's v3.6.3 sample carries a DZ4Team signing identity (personal name withheld). We did not diff the appsuite library in 3.6.3, so this is a chain of class and string names, not a per-version binary diff at every hop.

Enrollment chain, reproduced against lab stand-ins (confirmed in lab)

The chain below was reproduced offline. Every server value (the module, the verdict, the peer list) was supplied by us, because all of the operator's own discovery channels were dead at analysis time. The IMSDK-to-AltPop hand-off was performed by a module we wrote, since no operator-served module was ever observed (the live module list was empty).

1. A HopMon consent dialog appears. Our lab driver taps it. We did not capture the dialog's exact text or a screenshot; the app log records a single consent-gated activation, and the briefing's "single tap" refers to that.

2. `GET /active.php` returns `OK`, and an IMSDK worker queries `imsdk[.]tv/sdk/latest/info` for a module list.
3. A module JAR is downloaded and loaded by `DexClassLoader` with no integrity check beyond TLS validity for the named URL (the HopMon seed and API client accepts any certificate), cached under `app_dex/`. This code-loading channel is separate from the relay protocol. It loads unsigned code into the app's own process, which matters for the harm discussion below.
4. The AltPop engine runs a sixteen-site HTTP probe gate, then round-robins the four hardcoded `lb.<domain>` entries over `pixellog[.]io`, `ginuary[.]com`, `dnetk[.]com` and `newcommreview[.]com`, and (observed in lab) rejects private-looking DNS answers.
5. An mbedTLS client verifies the chain and hostname, then issues `GET /initreq/?os=android&ver=<sdk>&manufactor=<lc>&model=<lc>`, with non-alphanumerics percent-encoded. The device is treated as valid only if the response is HTTP 200 with a body of exactly `YES` (three bytes). This was pinned in disassembly at `libalt` 0x11bf64, and tested with a control: `YES\n` and `yes` both evaluate false. The verdict is cached in `neunative.txt` (the 24-hour reuse window is from a decompile read, not re-verified).
6. `GET /regdev?usr=<publisher>&userid=<uuid>&dev_ip=<ip or empty>&sdkv=12.0.30u&...` The client parses a JSON reply with `peer_servers` and geography keys (`dev_country`, `dev_state`, `dev_city`, `dev_asn`). We never saw a real operator reply; the geography and ASN are values the server would supply, not values the device measures.
7. The relay hostname is built as `<peer-ip>.<lb-domain>`, the node connects over TLS, and sends a REGISTER message.

The REGISTER message captured in the lab was 290 bytes, a nested TLV carrying the device UUID, country, city, state, publisher, engine version string (`12.0.30` plus `fg` in foreground, plus `u`), the `dev_asn` value echoed back from `/regdev` (field 0x6055, confirmed by varying the lab reply, which changed the message length from 290 to 295 bytes), carrier, device model, network type, Android SDK level, the host-app package as JSON, and a constant field of value 3 (read in code as a protocol-version constant). The 290-byte size is specific to the lab inputs; real sizes vary.

Relay protocol (high-confidence; code-read with partial lab confirmation)

The node's decoder compares exactly six inbound message types: RegisterResponse, Ping (the node replies Pong), OpenTunnel (tunnel id, host, port, and a TCP-or-UDP flag), Tunnel data, CloseTunnel, and Goodbye. The node also sends its REGISTER and a status message. OpenTunnel has no destination filter in the code read (this specific "no filter" point was not independently re-verified). There is no cryptography or signing beyond TLS, and no code, update or shell command in the relay channel; code loading lives in the separate IMSDK module system.

Observed in the lab: after a RegisterResponse the node sits idle holding the TLS connection, with no keepalive, and when the relay drops the connection the node reconnects within about a second to the same relay, with no new `/initreq` or `/regdev`. The code also contains a longer disconnect branch (a 60-second wait, then re-running the lb step), but that branch was not exercised, and the timer constants were not re-verified.

The HopMon egress filter (high-confidence, static only)

`libhopmn.so` version 0.9.3.2 (arm64 and x86_64 checked) has a single outbound-dial function that, before it opens a socket, rejects any destination in `10.0.0.0/8`, `127.0.0.0/8`, `172.16.0.0/12`, `169.254.0.0/16` or `192.168.0.0/16`, checked at dial time on the resolved address. The engine is IPv4-only.

Scope matters. This bounds the HopMon path specifically. It does not cover:

- The AltPop relay (`libalt`), where OpenTunnel has no filter found.
- Code delivered through the IMSDK `DexClassLoader` channel, which runs in the app's own process and could reach the LAN regardless.
- CGNAT (`100.64.0.0/10`), `0.0.0.0/8`, other special ranges, or IPv6.
- Other publisher builds or a future config; the filter is compiled into this build.

The claim is narrow: the HopMon proxy, in this build, will not dial a home-LAN, loopback or link-local IPv4 address. It is not a general statement that the app cannot reach a local network. This was read from the binary and never exercised dynamically.

One note on method: the first two passes over `libhopmn` (the resolver and the request handlers) showed no filter and nearly produced a wrong "no filter" conclusion. The filter lives in the dial function, reached through a function pointer with no direct cross-reference, and the verdict held only after confirming the constants in both builds.

Live run (confirmed, single run)

An unmodified Flix Vision 3.9.0, with the consent dialog shown and tapped by our lab driver, run once through an Oracle exit VM. AltPop itself did not run: the IMSDK controller answered `active INACTIVE`. HopMon did. The log shows `start() requested MoneytiserService`, then `HopmnProxy: Library loaded`, then a connection to `102[.]165[.]4[.]65:15101`, held for the ten-minute window.

Over the connection the controller pushed plaintext HTTP CONNECT jobs: 47 in total (45 CONNECT, one GET, one POST), reaching 39 distinct customer-requested hosts once the operator's own two ipinfo geolocation probes are excluded (the full list is in the lab record). The operator's two ipinfo probes arrived first, about 223 seconds after capture start; the first customer job came at about 382 seconds, so the exit carried no customer traffic for roughly the first six minutes. The requested hosts were mixed, consistent with both automated and human clients: Apple device services, several Cloudflare Turnstile endpoints (sites protected against automation), retail and news sites, a wiki, a media CDN, and one POST to `62[.]112[.]11[.]118:8082` whose purpose is unattributed (a check-in callback, a bot beacon and a reseller health probe all remain open, and the same URL was submitted to urlscan a week earlier).

Two points for accuracy. The controller address `102[.]165[.]4[.]65:15101` arrived in the app's config from the operator API, not from the APK, and whether that gateway is run by the SDK operator or a reseller is unknown; `MoneytiserService` is only the app's internal service name, not an attribution. And our monitoring of the operator's `active.php` endpoint returning `INACTIVE` reflects only the IMSDK module gate, not the HopMon proxy,

which is why the proxy ran while AltPop stayed dormant. On consent timing: the consent-gated imsdk activation (logged 02:22:40.186) and the MoneytiserService start (logged within the same roughly 20 ms window) do not resolve strict sub-second ordering, so the logs place HopMon starting at the consent point rather than cleanly after it; the driver tapped the dialog before this.

Harm-minimisation for this run: a single datacenter exit (not a residential one), a hard ten-minute cap, a killswitch, no content inspection beyond hostnames, and ongoing monitoring of the Oracle account for abuse reports. The captures contain third parties' traffic, some of it plain HTTP, and are kept local and never published.

SECTION 3

Infrastructure and pivots

Pivot graph: an export of the Muninn investigation graph (an internal link-analysis dataset) for this case, promoted edges only. No personal names appear in the infrastructure.

Indicator	Pivot type	Linked to	Confidence	Evidence
pubs[.]abnetworks[.]io + pubs[.]myrc[.]xyz	shared TLS cert (2026-09-12)	one operator	high-confidence	one CT certificate lists both exact names; both hardcoded as app seeds
pubs[.]abnetworks[.]io	CNAME	pubs[.]myrc[.]xyz	confirmed	direct DNS and DoH query 2026-10-22; also OTX passive DNS 2026-08-30
abnetworks[.]io / ai-data-solutions[.]com	shared Base44 App-Id 696528a884e402a6e24ef234	one storefront owner	confirmed (shared app-id) / high-confidence (owner link)	identical app bundle and X-App-Id on all three; the owner-binding inference is from Base44's model, not independently cited
abnetworks[.]ai and moishinexus[.]com	shared M365 tenant TXT (NETORGFT20878876)	same mail tenant	confirmed (fact) / working hypothesis (same operator)	identical tenant verification record; abnetworks[.]io is a separate tenant
102[.]165[.]4[.]65	passive DNS	nn/ext.myrc[.]xyz, gw.lb-viprox[.]net	working hypothesis	single-source VT passive DNS, 2023 to 2024
myrc[.]xyz	VT communicating files	Ocean Streamz (streamzy.com.ocean)	working hypothesis	81 of 84 VT files; SDK embedding not verified; the build we pulled had no SDK
91[.]134[.]10[.]83	live TLS SNI	pubs[.]abnetworks[.]io	confirmed	observed in the live-run capture

Strongest chain, and its weakest hop. Flix Vision hardcodes `pubs[.]abnetworks[.]io` and `pubs[.]myrc[.]xyz`. One certificate from 2026-09-12 lists both exact names, and `pubs[.]abnetworks[.]io` is a CNAME to `pubs[.]myrc[.]xyz`. The three abnetworks domains serve one private Base44 application, which only the owner can bind to a custom domain, so that is operator identity rather than shared hosting. The live proxy traffic went through `102[.]165[.]4[.]65`. The hop that joins the gateway to this domain cluster is single-source VT passive DNS from 2023 to 2024, and the gateway's config arrived from the operator API, not from a seed we traced. So the app-to-ABNetworks tie is a chain of inference, strong at the seed-domain end and thin at the gateway end. It is not a leaked contract.

What is not a fingerprint. The `ss-product-id: aab-v1` header first looked like an operator marker, but it appears on 17 to 18 unrelated GoDaddy small-business sites; it is a GoDaddy managed-hosting platform header, and the idea of pivoting on it is retracted. The shared Render IP across the three abnetworks domains is weak on its own, because Render is a shared platform; the Base44 App-Id carries that link, not the IP. Privacy protection on every domain is a pattern, not an identifier. The Ras Al Khaimah address the vendor lists is a free-zone agent building shared by many companies, so no link is drawn to an unrelated company registered there.

Negatives, recorded. No organisation name appears in any certificate across the cluster. CT shows only `in.`, `pubs.` and `www.` subdomains for `abnetworks[.]io`, and a wildcard for `moishinexus[.]com`, with no new siblings. There is no breach or paste hit on the three published emails (manual lookups; the services and a known-breached control are not documented, so treat this as "not found," not "clean"), and no LinkedIn footprint. A backend-fingerprint test came back negative: `moishinexus[.]com` and the abnetworks sites run on separate code stacks, so there is no web-layer link between them, only the shared mail tenant. The Base44 and tenant internals are behind authentication and were not pursued.

On the Moishi Nexus link specifically. The fact is that `abnetworks[.]ai` and `moishinexus[.]com` publish the same Microsoft 365 tenant record. The hypothesis is one controlling party. Alternatives that the evidence does not rule out: one tenant administered by a managed-service provider or consultant for two clients (Moishi Nexus itself advertises managed IT services), which would produce the same record. The claim "unique per tenant" rests on GoDaddy's naming convention for resold Microsoft 365, not on vendor documentation. We did not corroborate the link at any other layer, and the web backends are different.

What would break the link. If the Base44 App-Id were shared across clearly unrelated customers, the three-domain tie weakens (a per-app private id makes that unlikely). If the 2026-09-12 certificate were a shared-hosting SAN bundle rather than an operator-requested one, the abnetworks-to-myrc tie weakens (it lists exactly the two seed names, which argues against that). If the shared tenant were shown to be administered by a managed-service provider for two clients, the same-operator reading of Moishi Nexus falls away.

SECTION 4

Comparison to prior campaigns

This is a continuation of the Popa/HopMon operation documented by Synthient and Qurium, with the same engine family renamed and the same consent-and-harvest model. It sits in the broader proxyware space alongside other reported SDKs and apps (for example ProxyLib/LumiApps, documented by HUMAN's Satori team, and IPRoyal Pawns) and, at the malicious end, proxy botnets such as Socks5Systemz; these are different things, grouped here only as market context and not as equivalents. The surrounding residential-proxy resale market is noted as context; this issue does not primary-source any specific reseller's relationship to this SDK.

SECTION 5

IOC reference

All indicators defanged.

Type	Indicator	Note
IPv4	102[.]165[.]4[.]65	live proxy controller, port 15101 (Tier.Net / AFRINIC DET Africa); operator vs reseller unknown
IPv4	23[.]227[.]195[.]x	operator seed/API IPs that supplied the live controller config (observed via the app's seed lookup)
IPv4	91[.]134[.]10[.]83	seed host (OVH): pubs/test.myrc, pubs.abnetworks
IPv4	198[.]24[.]188[.]50, 198[.]24[.]187[.]90	myrc / lb-viprox hosting (Secured Servers)
IPv4	66[.]165[.]245[.]150, 66[.]206[.]4[.]210	myrc subdomain hosting (Hivelocity)
IPv4	216[.]24[.]57[.]1	ABNetworks storefront host (Render, shared PaaS; context only, do not block)
IPv4	62[.]112[.]11[.]118	customer-side target of one relayed POST, port 8082; not operator infrastructure
Domain	abnetworks[.]io, abnetworks[.]ai, ai-data-solutions[.]com	ABNetworks storefront, one Base44 app
Domain	moishinexus[.]com	second brand sharing a mail tenant with abnetworks[.]ai
Domain	myrc[.]xyz and subs pubs, pubs2, r, nn, ext, cust, distribution, ovh, test	seed and panel domain for whoever controls the cluster
Domain	pubs[.]abnetworks[.]io, in[.]abnetworks[.]io	abnetworks seed / unexamined subdomain
Domain	lb[.]pixellog[.]io, lb[.]ginuary[.]com, lb[.]dnetk[.]com, lb[.]newcommreview[.]com	AltPop load-balancer domains; on Shadowserver sinkholes since July to August 2026 (hits will include researchers)
Domain	lb-viprox[.]net, gw[.]lb-viprox[.]net, direct[.]lb-viprox[.]net	relay-adjacent domains (gateway link is single-source passive DNS)
Domain	stars[.]apkdatamod[.]com	AltPop CNAME-discovery fallback name (dead on 2026-10-19; cause not established)
Domain	imsdk[.]tv, seed1-943u092[.]ddns[.]net, seed2-943u092[.]ddns[.]net	IMSDK module controller and seed servers
SHA256	3a4fffd1838f0000e42d0aa7d61e3f0e5b014e815c278e98298eed2f1ecb8dd1	FlixVision v3.9.0.apk (seeds and HopMon examined on this build)
SHA256	9620500bd77a6ff6656ee54fff1150eb8e47f85e51024453a49d2cd5fb44d0b6	FlixVision v3.8.0.apk
SHA256	0408be82483cffc20b0dd69d84e076f29f3fff187273161eedb385b2e7d9917a	FlixVision v3.7.0.apk

Type	Indicator	Note
SHA256	d32d384b5910df49b049d8c8a1d422363091f4001f2b0fe48282f6c83313bbb3	libalt.so (AltPop 12.0.30), arm64
SHA256	487f2c56d9774635eea28b391599cfc08b9fa34b9e3edcb77cccb7894f1e81e3	libalt.so (AltPop 12.0.30), x86_64
SHA256	6dcac2ef9d2887ece65f5252a2cfb27a2f2cadf703480d77ef6c5f414e8aebce	libhopmn.so (hopmnproxy-0.9.3.2), arm64
SHA256	9fc893ad17086236b598fe652ee8df540ccd0f863c512aa37902dd00ebe8da10	libhopmn.so (hopmnproxy-0.9.3.2), x86_64

SECTION 6

Detection

Two YARA rules (validated with yara 4.5.8: clean compile, matched both architecture builds of each library, silent on the older X-Plore engine and on an unrelated control binary; no goodware-corpus run, so the false-positive surface is reasoned, not measured) and two Sigma rules (field names from the Sigma proxy and DNS taxonomy; not yet run through a pySigma backend). Note that the AltPop network indicators point at sinkholed infrastructure, so matches there will include researchers and scanners. The live component was HopMon, so a HopMon network rule is included.

```
rule AltPop_SDK_libalt
{
    meta:
        description = "AltPop residential-proxy SDK native engine (libalt.so). Seen in Flix Vision 3.7.0 to 3.9.0. Shares codebase lineage with the Popa/HopMon io.nn.alpha engine."
        author = "Bornov Shyam Kalita"
        date = "2026-10-22"
        reference = "GhostWire Issue 004 (2026-BSK-004)"
        hash = "d32d384b5910df49b049d8c8a1d422363091f4001f2b0fe48282f6c83313bbb3"
        confidence = "confirmed for this generation; the JNI anchor is generation-specific and will miss a future rename"
    strings:
        $jni     = "Java_com_ax_altpop_sdk_1actions_AltSdkEngine_" ascii
        $initreq = "/initreq" ascii
        $ver     = "12.0.30" ascii
        $peer    = "peer_servers" ascii
        $man     = "manufactor" ascii
    condition:
        uint32(0) == 0x464c457f and filesize < 8MB
        and $jni and 2 of ($initreq, $ver, $peer, $man)
}

rule HopMon_proxy_libhopmn
{
    meta:
        description = "HopMon host-app proxy server (libhopmn.so, self-identified hopmnproxy-0.9.3.2, looks 3proxy-derived). Runs the bandwidth-sharing exit."
        author = "Bornov Shyam Kalita"
        date = "2026-10-22"
        reference = "GhostWire Issue 004 (2026-BSK-004)"
        hash = "6dcac2ef9d2887ece65f5252a2cfb27a2f2cadf703480d77ef6c5f414e8aebce"
        confidence = "confirmed for this build; the banner regex is pinned to the 0.9.x family"
    strings:
        $banner = /hopmnproxy-0\.9(\.[0-9]+)+/ ascii
        $vanity = "hopmn allows to share and control Internet connection" ascii
    condition:
        uint32(0) == 0x464c457f and filesize < 4MB and $banner and $vanity
}
```

```
title: AltPop Proxyware Load-Balancer and Seed Lookup (DNS)
id: 7a3f6e2c-1b94-4d7a-9f21-2c8d5e0a4b6f
status: experimental
description: >
  DNS lookups for the AltPop/HopMon SDK load-balancer and seed domains. The four
  lb domains are sinkholed, so hits there include researchers and scanners. The
  seed domains (pubs.abnetworks.io, pubs.myrc.xyz) resolved live during analysis,
  so hits there are more likely real devices. Pair with an app inventory check for
  Flix Vision. imsdk.tv ownership is not established; treat its hits as context.
author: Bornov Shyam Kalita
date: 2026/10/22
references:
  - GhostWire Issue 004 (2026-BSK-004)
logsource:
  category: dns
detection:
  lb_and_seed:
    query|endswith:
      - '.pixellog.io'
      - '.ginuary.com'
      - '.dnetk.com'
      - '.newcommreview.com'
      - 'pubs.abnetworks.io'
      - 'pubs.myrc.xyz'
    query:
      - 'imsdk.tv'
  condition: lb_and_seed or query
falsepositives:
  - Security researchers, sinkhole operators, and sandboxes resolving these domains.
  - Threat-intel feeds and scanners.
level: medium

---
title: HopMon Proxyware Controller Connection
id: 2d9c4b11-8f3a-4e60-b7d2-1a6e9c0f5b42
status: experimental
description: >
  Outbound connection to the HopMon proxy controller observed in the Flix Vision
  live run. This is the component that was active, unlike the AltPop relay. The IP
  and port are from a single observation and may rotate.
author: Bornov Shyam Kalita
date: 2026/10/22
references:
  - GhostWire Issue 004 (2026-BSK-004)
logsource:
  category: firewall
```

```
detection:
  controller:
    dst_ip: '102.165.4.65'
    dst_port: 15101
  condition: controller
falsepositives:
  - The IP may be reassigned; confirm with the port and a long-lived session carrying plaintext HTTP CONNECT jobs issued to a mobile app process.
level: medium
```

SECTION 7

Limitations

- The link from Flix Vision to ABNetworks FZ-LLC is inference, from the hardcoded seed, the shared certificate, the CNAME, and the vendor's own "partner apps" wording. It is not a contract or a payment record, and the gateway-to-cluster hop is single-source passive DNS.
- The roles involved (SDK vendor, app publisher or repacker, proxy reseller, proxy customer) are not separated. "Operator" in this issue is shorthand for whoever controls the abnetworks domain cluster, which is not shown to be the SDK operator.
- The live run is a single run, one datacenter exit IP (not residential), on an x86_64 emulator. The first customer job arrived about 223 seconds in. We did not buy or test ABNetworks' product and have no evidence our exit IP appeared in it.
- The AltPop enrollment chain was reproduced with lab stand-ins; every server value was supplied by us, and the IMSDK-to-AltPop hand-off was done by a module we wrote. No operator-served module or live AltPop relay was ever observed. All AltPop discovery channels were sinkholed or dead at analysis time.
- The HopMon egress filter is static-only, scoped to `libhopmn` 0.9.3.2, and does not cover the AltPop relay, the module channel, CGNAT, or IPv6 (see that section).
- Direction and ownership of the code lineage are not established.
- The same M365 tenant between `abnetworks[.]ai` and `moishinexus[.]com` is a fact; "same operator" is a hypothesis with a shared-IT-provider alternative.
- Ocean Streamz contacting `myrc[.]xyz` is a VirusTotal relationship only; the build we pulled carried no SDK.
- Several timing and protocol details (the 60-second reconnect branch, the 24-hour cache window, the "no destination filter" in OpenTunnel) are code reads not re-verified dynamically.
- No right of reply or coordinated disclosure to ABNetworks, Moishi Nexus, Base44, Render or GoDaddy has been done yet. That step should precede publication.

SECTION 8

References

- Krebs on Security, "FBI Seizes NetNut Proxy Platform, Popa Botnet" (2026-07-02). https://krebsonsecurity.com/tag/netnut/
- Synthient, Popa proxyware research (2026; the X-Plore carrier and the NetNut-forwarding assessment). https://synthient.com [confirm exact permalink from Issue 003 references]
- Qurium, "Finding Popa: When Your Smart TV Stops Being Yours" (2026-06-18). https://www.qurium.org/forensics/finding-popa/ ; and "Op NutCracker: FBI Seizes Domains Used by NetNut's Residential Proxy Network." https://www.qurium.org/forensics/op-nutcracker-fbi-seizes-domains-used-by-netnuts-residential-proxy-network/
- GhostWire Issue 003, X-Plore and the Popa SDK. https://ghostwireintel.page/report.html?id=003
- Cisco Talos, "Exploitation of proxyware." https://blog.talosintelligence.com/proxyware-abuse/ [verify on primary read]
- HUMAN Satori Threat Intelligence, ProxyLib / LumiApps. https://www.humansecurity.com/learn/blog/satori-threat-intelligence-alert-proxylib/ [verify on primary read]
- Internal lab and RE record (not for publication): `research/{libalt_static, altpop_protocol_lab_derived, optionA_remote_module_proof, live_v1_unmodified, hopmn_egress_filter, abnetworks_corporate}.md`.